

Case Study Report: StockFlow Inventory Management

Project: B2B SaaS Inventory Solution

Platform: StockFlow

Scope: Backend Architecture, Database Design, & Operational APIs

Executive Summary

StockFlow is designed to empower small businesses by providing real-time visibility into their supply chain. This report outlines the transition of the core "Product & Inventory" module from a fragile prototype to a production-ready, multi-tenant system. We focus on three critical pillars: **Data Integrity**, **Relational Scalability**, and **Actionable Intelligence**.

Part 1: Code Review & Production Hardening

1.1 The Prototype Audit

The original `create_product` endpoint suffered from several "silent failures" that would cause operational friction in a live B2B environment:

- **Transactional Fragility:** The code committed to the database twice. If the product was created but the inventory update failed, the system was left in an inconsistent state (a product with no record of where it is stored).
- **Model Misalignment:** It assumed a product belongs to one warehouse. In reality, modern B2B logic requires products to exist across a network of locations.
- **Lack of Validation:** The API accepted negative prices and missing fields, which leads to "garbage data" that breaks downstream accounting and analytics.
- **Multi-tenant Blindness:** There was no verification that the user had the authority to add products to the specified warehouse.

1.2 Impact in Production

- **Loss of Trust:** 500-level errors and duplicate SKUs make the software feel unreliable to paying customers.
- **Data Corruption:** Incorrect pricing or stock levels can lead to overselling or financial discrepancies.

1.3 The Solution: Production-Ready Endpoint

The revised implementation uses a **single transaction block** and robust input sanitization.

```
@app.route('/api/products', methods=['POST'])
```

```
def create_product():
```

```
    data = request.get_json() or {}
```

```
    # 1. Strict Validation
```

```
    required = ['name', 'sku', 'price']
```

```

if not all(k in data for k in required):
    return jsonify({"error": "VALIDATION_ERROR", "message": "Missing required fields"}), 400

try:
    price = float(data['price'])
    quantity = int(data.get('initial_quantity', 0))
    if price < 0 or quantity < 0:
        raise ValueError
except (TypeError, ValueError):
    return jsonify({"error": "VALIDATION_ERROR", "message": "Invalid numeric values"}), 400

```

2. Atomic Database Operations

```

try:
    # Decouple Product from Warehouse (Many-to-Many preparation)
    new_product = Product(
        name=data['name'],
        sku=data['sku'],
        price=price
    )
    db.session.add(new_product)
    db.session.flush() # Fetch ID for the join table

    warehouse_id = data.get('warehouse_id')
    if warehouse_id:
        # Verify warehouse exists before linking
        if not Warehouse.query.get(warehouse_id):
            return jsonify({"error": "NOT_FOUND", "message": "Warehouse not found"}), 404

    inventory_record = Inventory(
        product_id=new_product.id,
        warehouse_id=warehouse_id,

```

```

        quantity=quantity
    )
    db.session.add(inventory_record)

    db.session.commit()
except IntegrityError:
    db.session.rollback()

    return jsonify({"error": "CONFLICT", "message": "SKU already exists"}), 409

return jsonify({"message": "Product created", "product_id": new_product.id}), 201

```

Part 2: Database Architecture

2.1 The Philosophy of the Schema

The StockFlow database is designed for **Auditability** and **Growth**. We utilize a normalized structure to ensure that a single change (like updating a supplier's email) propagates correctly across the entire system.

2.2 Table Definitions (Highlights)

Table	Purpose	Key Constraint
Companies	Top-level tenant container.	PK: id
Products	The "Definition" of an item.	Unique: sku
Inventory	The "Physical State" of an item per location.	Unique: (product_id, warehouse_id)
Movements	Immutable ledger of every stock change.	FK: inventory_id
Bundles	Maps "Parent" products to "Component" parts. PK: (bundle_id, component_id)	

2.3 Strategic Decisions

- **Inventory Movements:** We never just "overwrite" stock levels. Every change is logged in `inventory_movements`. This allows us to generate "Stock over Time" reports and investigate discrepancies (e.g., shrinkage vs. sales).
- **Safety Stock Overrides:** While product types have default thresholds (e.g., Simple = 20), individual warehouses can override these. A warehouse in a remote location might require a higher safety buffer than a central hub.

Part 3: Operational Intelligence (Low-Stock API)

3.1 Turning Data into Actions

The Low-Stock API doesn't just list low numbers; it identifies **risks**. By combining inventory levels with "Recent Sales Activity," we prioritize alerts for items that are actually moving.

3.2 Implementation Strategy

We calculate the **Effective Threshold** (the point at which an alert triggers) by checking for specific overrides first, then falling back to general product defaults. We also calculate an estimated **Stockout Date** to help managers prioritize their reorders.

snippet of the logical flow

```
effective_threshold = func.coalesce(
    Inventory.safety_stock_threshold,
    func.case(
        (Product.product_type == 'BUNDLE', 10),
        else_=20
    )
)

# Filtering for active items only
query = db.session.query(...).filter(
    Inventory.quantity < effective_threshold,
    Product.id.in_(recent_sales_subquery) # Only alert if people are buying it
)
```

3.3 Handling Edge Cases

- **The "Silent Seller":** If a product is low on stock but hasn't sold in 30 days, it is excluded from the primary alert list to prevent "Alert Fatigue."
- **Supplier Gaps:** If an item is low but has no assigned supplier, the API returns a null supplier object, signaling the UI to prompt the user for an assignment.
- **Multi-tenant Isolation:** Every query is strictly scoped to the company_id. Even if a user knows a product ID from another company, the database joins will return no results, ensuring total data privacy.

Final Recommendations for Product Team

1. **Global vs. Local SKU:** We should clarify if a SKU is unique across the whole platform or just within a company. Current design assumes global uniqueness.
2. **Seasonality:** The days_until_stockout calculation is currently a linear average. We should consider a "Weighting" system where sales from the last 7 days count more than sales from 30 days ago.
3. **Real-time Webhooks:** Instead of users polling the Alerts API, we should implement Webhooks to notify managers via Slack or Email the moment a "High Velocity" item hits its threshold.

